

Agentic AI for Games

From language models to tools, ReAct and MCP

Simon Lucas // (aided by Codex, ChatGPT Sol and Astra)



Code and examples

Tutorial snapshot: 14 September 2026

Python examples: exact counting → search → gameplay, PCG

Why give an LLM tools?

Language models are useful for

- Interpreting a task and its rules.
- Proposing actions and strategies.
- Choosing which capability to use.

Code supplies concrete capabilities

- Exact arithmetic and counting.
- Access to current game state.
- Simulation, search and validation.

Let the model choose an operation; let tested code perform it.

Tools can improve reliability. Choosing the wrong tool or arguments can still fail.

Start with an ordinary Python function

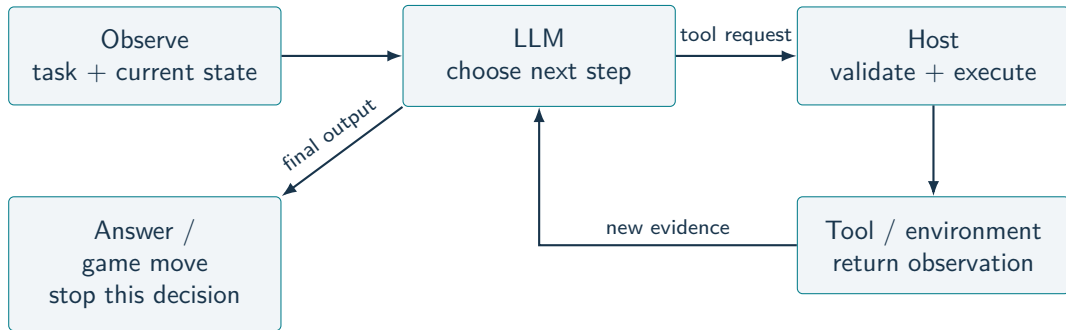
```
def multiply(a: float, b: float) -> float:  
    """Multiply two numbers."""  
    return a * b
```

src/simple_examples/operations.py

- Calling `multiply(6, 7)` returns 42.
- No language model or protocol is needed for this calculation.
- The agentic step is deciding *when* to call it from a language request.

Separate the capability from the machinery that invokes it.

ReAct: reason, act, observe, repeat



ReAct interleaves reasoning and actions, using observations to revise the next step. Our function-calling loop follows this pattern; it does not require a printed “Thought:” transcript or expose a model’s private reasoning.

Yao et al., ReAct, ICLR 2023. <https://arxiv.org/abs/2210.03629>

One concrete interaction

1. **Task:** “What is six times seven?”
2. **Model action:** request `multiply(a=6, b=7)`.
3. **Host action:** execute the tool and collect 42.
4. **Observation:** append the result to the conversation.
5. **Model answer:** “42.”

A tool call is a structured request. The host runs the code.

Stop on a final answer; bound the number of calls and handle failures.

Why introduce MCP?

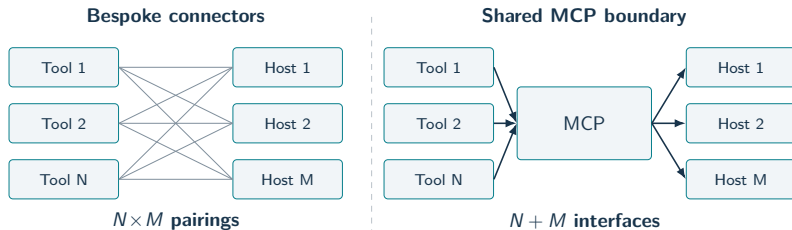
Model Context Protocol: a shared interface to capabilities.

- Discover tool names, descriptions and input schemas.
- Invoke a tool with structured arguments; receive its result.
- Reuse a server across compatible host applications.

ReAct describes the loop. MCP connects the loop to tools.

Direct Python calls remain sufficient inside one application. MCP adds a reusable process/service boundary. It does not make a tool or model correct.

MCP changes the integration shape

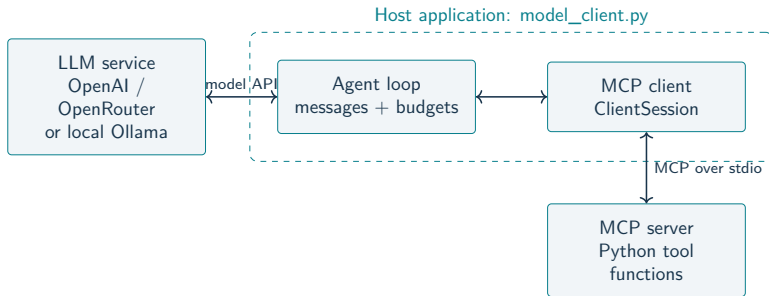


- Bespoke connectors: N tool providers and M agent host applications can require up to $N \times M$ integrations.
- MCP boundary: implement each provider once and each host once—roughly $N + M$ interfaces.
- The host-model API remains separate; the host controls which tools the model sees.

Standardize the boundary so tools and hosts can vary independently.

Authentication, deployment, permissions and useful schemas still require engineering.

The architecture in this repository



- A **host is the coordinating application**, not a physical machine.
- Here, `model_client.py` owns the agent loop and MCP client. The `.sh` script loads configuration and launches that host.
- The host starts the MCP server over stdio. With Ollama, all three can run locally; with OpenAI/OpenRouter, only the model service is remote.

MCP architecture, specification 2025-06-18; Python SDK implementation.

Expose the function as an MCP tool

```
@mcp.tool()
def multiply(a: float, b: float) -> float:
    """Multiply two numbers and return the result."""
    return multiply_numbers(a, b)
```

src/simple_examples/server.py

- `@mcp.tool()` registers a callable capability.
- Type hints describe inputs; the docstring explains its purpose.
- The server starts with `mcp.run(transport="stdio")`.

With stdio, keep diagnostic output off stdout: it carries protocol messages.

Type hints become a JSON input schema

```
@mcp.tool()
def multiply(
    a: float,
    b: float
) -> float:
    """Multiply two numbers
    and return the result."""
    return multiply_numbers(a,
        b)
```

Actual inputSchema from list_tools

```
{
  "properties": {
    "a": {"title": "A", "type": "number"},
    "b": {"title": "B", "type": "number"}
  },
  "required": ["a", "b"],
  "title": "multiplyArguments",
  "type": "object"
}
```

FastMCP derives the name, description and schema when the tool is registered.

The model receives a machine-readable contract, not the Python body.

Run: `uv run python src/simple_examples/mcp_client.py -schema multiply`

Exact counting: how many r's in strawberry?

LLM alone

- Generates an answer from tokens and context.
- Observed OpenRouter answer: *The letter r occurs 2 times.*
- Output can vary; there is no independent computation to inspect.

LLM + MCP tool

```
count_letters(  
    text="strawberry",  
    letter="r"  
)  
-> 3
```

The trace records the operation and arguments; tested Python performs the count.

Tools turn a brittle language task into an exact, repeatable operation.

```
./src/simple_examples/scripts/compare_strawberry.sh openrouter  
# Also accepts: openai or qwen
```

One observed run, 17 September 2026; raw model responses can vary.

Same decorator, different variable names

```
# simple_examples/server.py
mcp = FastMCP("beginner-arithmetic-and-strings")
@mcp.tool()
def multiply(a: float, b: float) -> float:
    ...

# griddle/search_server.py, inside create_server()
server = FastMCP("griddle-search", log_level="WARNING")
@server.tool()
def mcs_advice(rollouts_per_square: Budget = 10) -> dict:
    ...
```

Abbreviated repo excerpts. Both objects are FastMCP instances.

Register on the chosen server object; its variable name is arbitrary.

`@mcp.tool()` means `function = mcp.tool()(function)`. It registers the function when this definition executes; it does not run it.

MCP decorators: operation, context or template?

Decorator	When to use it
<code>@mcp.tool()</code>	An operation: compute, search or act.
<code>@mcp.resource(uri)</code>	Addressable context the application can read, such as game rules.
<code>@mcp.prompt()</code>	A reusable, user-selectable prompt template; it does not call the model.
<code>@mcp.completion()</code>	Suggestions for prompt/resource arguments.

This repository registers tools only. The host decides which to offer to the LLM.

Griddle's `configure` and `set_position` are registered tools, but excluded from the model's allowlist. Registration is not a privacy boundary.

Other decorators have different jobs

Decorator	Purpose in this repository
<code>@dataclass(frozen=True, ...)</code>	Pydantic validation; frozen fields.
<code>@model_validator(...)</code>	Cross-field checks, e.g. unique tool names.
<code>@asynccontextmanager</code>	Acquire and close clients around <code>async with</code> .
<code>@classmethod</code>	Alternate constructor: <code>new_game(cls, ...)</code> .
<code>@staticmethod</code>	Class helper without implicit <code>self</code> : <code>_rollout</code> .
<code>@property</code>	Computed attribute: <code>match.score</code> .

These organise Python behaviour; none exposes an MCP tool.

Decorators apply when definitions execute. Read the import or owning object to understand what each one does.

`docs/decorators.md`; `src/game_agent/griddle/`

Discover and call a tool without an LLM

```
async with stdio_client(server) as (read, write):  
    async with ClientSession(read, write) as session:  
        await session.initialize()  
  
        tools = await session.list_tools()  
        print("Discovered tools:", ", ".join(tool.name for tool in tools.tools))
```

src/simple_examples/mcp_client.py

Test the protocol and tools independently before adding model behaviour.

Bridge MCP schemas to the model API

```
return [  
    {  
        "type": "function",  
        "function": {  
            "name": tool.name,  
            "description": tool.description or "",  
            "parameters": tool.inputSchema,  
        },  
    }  
    for tool in mcp_tools  
]
```

src/simple_examples/model_client.py : as_model_tools

- The model sees a tool's name, purpose and argument schema.
- It does not receive the Python function body.
- The host translates between the two interfaces.

The loop: ask the model for its next step

```
for _ in range(8):
    response = await llm.chat.completions.create(
        model=model, messages=messages, tools=tools, tool_choice="auto"
    )
    message = response.choices[0].message
    messages.append(message.model_dump(exclude_none=True))

    if not message.tool_calls:
        return message.content or "(The model returned no text.)"
```

src/simple_examples/model_client.py : answer (excerpt)

`tool_choice="auto"` allows either a tool request or an answer. The eight-round bound prevents an unending loop.

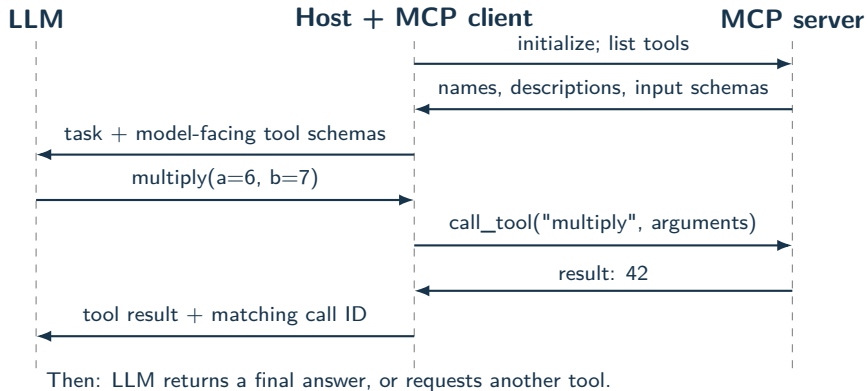
The loop: execute and return the observation

```
for call in message.tool_calls:
    arguments = json.loads(call.function.arguments)
    print(f"MCP call -> {call.function.name}({arguments})")
    result = await mcp_session.call_tool(call.function.name, arguments)
    if result.isError:
        raise RuntimeError(tool_result_text(result))
    messages.append(
        {
            "role": "tool",
            "tool_call_id": call.id,
            "content": tool_result_text(result),
        }
    )
```

src/simple_examples/model_client.py : answer (excerpt)

The call ID links each observation to the request it answers.

The same interaction as a sequence diagram



From arithmetic to Griddle

- Place 25 sequential letters on a 5×5 grid.
- Each placement is permanent; future cards are hidden.
- Score contiguous words left-to-right and top-to-bottom.
- Overlaps and subwords count; one-letter words do not.

Word length	Points
2	1
3	3
4	7
5	10

This implementation: 73-card deck; bundled dictionary is authoritative.

The objective is expected final score, not just an immediate word.

Monte Carlo Search (MCS) as a tool

```
@server.tool()
def mcs_advice(rollouts_per_square: Budget = 10) -> dict:
    """Compare every empty square with full random game completions.

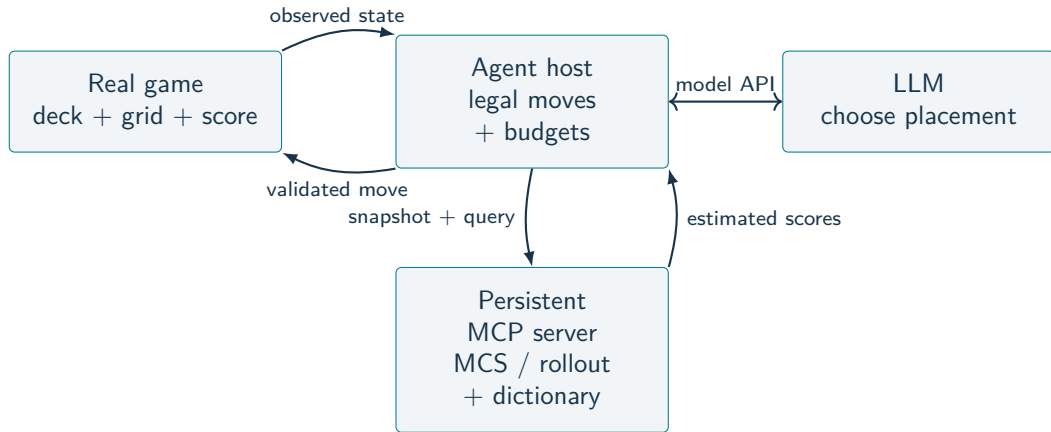
    Returns a recommended absolute cell index and estimated final scores.
    Future letters are sampled, never the actual hidden deal. Does not place a
    letter.
    """
    return context.mcs_advice(rollouts_per_square)
```

src/game_agent/griddle/search_server.py

For each empty square: place the letter in a simulation, complete the game randomly, average the final scores. Repeat B times per square.

At 25 empty squares and $B = 10$: 250 simulated completions. These are noisy estimates, not guaranteed outcomes.

Keep advice separate from real game actions



The host supplies the current game state. Search returns advice; the LLM selects an index; the host validates and applies the move. The search server never mutates the real board.

Reuse resources; enforce limits

- One HTTP client and one MCP process per game.
- Discover tools and configure the dictionary once; update each position.
- Expose only selected search tools to the model.
- Cap rollouts and calls; reject illegal moves with bounded corrections.

Host-only `configure` and `set_position` are never offered to the LLM.

```
src/game_agent/griddle/llm_agent.py : game_session, _choose, _decision_loop
```

A fresh model conversation is used for each placement; process persistence is not conversation memory.

Evaluate the whole policy

- Use the same deal seeds; record model, prompt, provider and search budget.
- Compare random, standalone search, LLM alone and LLM + search.
- Measure score, time, tokens, invalid moves and actual tool use.
- Preserve full traces: did the LLM follow or override the advice?

Tool access, tool use and useful tool use are different questions.

Fixed deals do not guarantee deterministic hosted model responses. Three games are a demonstration, not a robust model ranking.

Small experiment: search makes the difference

Policy	Mean score	Seconds/game
Random	27.3	< 0.01
Standalone MCS-10	84.7	0.60
Standalone MCS-200	93.7	11.88
GPT-4o-mini, no tools	25.3	23.30
GPT-4o-mini + persistent MCP MCS-10	86.3	44.49
GPT-4.1, no tools	24.7	34.92
GPT-4.1 + persistent MCP MCS-10	67.7	54.60

Three deals (0–2), OpenRouter, original rules prompt. Standalone and tool search use different simulation seeds. GPT-4o-mini followed all 72 search recommendations: this shows useful delegation, not an advantage over standalone search.

Separate, incomplete Sol medium-reasoning trials took 26–28 minutes for one no-tool game; they are excluded from this three-deal table.

Completed three-deal runs; full reports and caveats: `docs/griddle_llm.md`.

Latency: measure before blaming the tool

Same moves and scores

Recreated each move: **57.73 s/game**

Reused for each game: **44.49 s/game**

Observed reduction: **22.9%**

Persistent run: mean seconds/game

LLM requests	42.68
Search computation	1.02
MCP startup/discovery	0.46

Most elapsed time was in model requests, not Monte Carlo search.

Provider latency varies. This rerun changed both HTTP and MCP lifetime; the timings do not isolate their separate contributions.

`results/griddle/llm_persistence_summary.json`

PCG objective: lengthen the shortest route

- A binary grid has open passages and walls; start is top-left, goal bottom-right.
- Breadth-first search supplies exact fitness: the shortest valid route's length.
- Mutation flips cells; a $(1 + 1)$ evolutionary loop retains non-worsening candidates.



One 20×10 run, sampled across 5,000 mutation attempts; blue marks a shortest route.

`results/pcg/maze_evolution.svg`

Maze fitness has a scale

Grid	Iterations	Scores (seeds 0–2)	Mean	Seconds/maze
12×8	2,000	38, 38, 38	38.0	0.09
12×8	5,000	42, 48, 48	46.0	0.20
20×10	2,000	70, 70, 66	68.7	0.17
20×10	5,000	82, 88, 86	85.3	0.41

Compare raw path-length fitness only at the same grid dimensions.

The interactive configuration was 20×10 with 5,000 iterations; the compact LLM benchmark uses 12×8 with 2,000. This accounts for scores near 80 versus 38. Times are local three-run means.

Reproduced seeds 0–2; method and commands: `docs/pcg_maze.md`.

One-shot generation: fast evolution, diverse LLM output



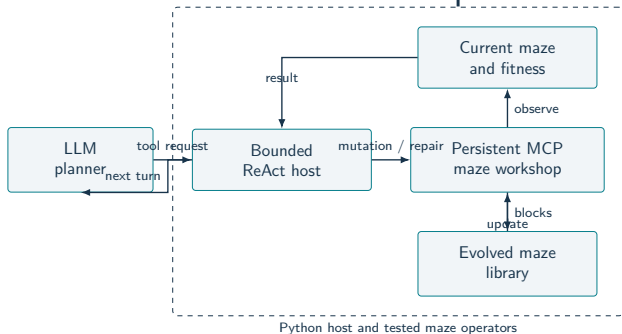
Six 12x8 mazes per approach: `results/pcg/maze_gpt5_mini_comparison.json`.

Approach	Valid	Fitness	s/maze
Random	4/6	18.0	< 0.01
Evolution-2000	6/6	39.7	0.08
GPT-5-mini	6/6	24.0	40.65

Set-level diversity

- Mean pairwise Hamming distance: random .304, evolution .463, LLM .506.
- Gzip size: 117, 186 and 104 bytes. Compression also reflects repeated text structure, so report both measures.

Agentic PCG: give the model a maze workshop



- Inspect fitness and validity; request batches of local mutations.
- Copy rectangular macro-patterns from an evolved maze library.
- Repair disconnected candidates, then select or restore earlier states.

Tools turn generation into an observable edit–evaluate–repair loop.

Related work: Jiang et al., Agentic PCG (2026): iterative level editing with environment feedback.

Agentic maze result: validity recovered



Approach	Valid	Fitness	s/maze
Evolution-2000	3/3	38	0.10
GPT-4o-mini one-shot	1/3	18	1.00
GPT-4o-mini agentic	3/3	28	8.97

The agentic run made 28 model calls and 24 successful tool calls through one persistent MCP process. Its mean pairwise Hamming distance was .479.

Compute caveat: each agentic maze first builds a six-member library with 400 evolutionary iterations each. It is not an equal-compute comparison with the 2,000-iteration baseline.

Three 12x8 mazes: `results/pcg/maze_agentic_comparison.*`

Try it: make one controlled comparison

Start locally—no credentials

```
./src/simple_examples/scripts/run_basics.sh  
./scripts/compare_griddle_agents.sh  
uv run evo-maze render --seed 7 --iterations 2000
```

Then copy a hosted experiment

```
cp configs/griddle/llm_comparison.json \  
  /tmp/my_griddle.json  
# Edit one value in /tmp/my_griddle.json  
./scripts/compare_griddle_llm.sh \  
  /tmp/my_griddle.json \  
  results/griddle/my_griddle.json
```

A useful experiment

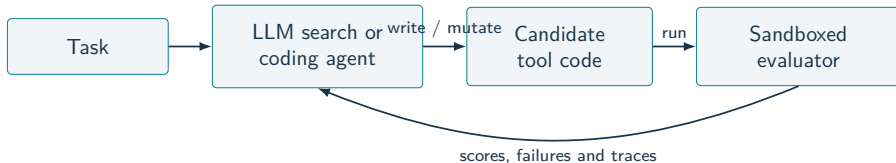
1. Choose Griddle or maze generation.
2. Keep seeds, rules and evaluator fixed.
3. Change one variable: model, prompt, tool or budget.
4. Compare score, validity/diversity, time and tool traces.
5. Repeat before drawing a conclusion.

Credentials stay local

Copy `.env.example` to `.env`. The root `.gitignore` explicitly excludes `.env` and `.env.*`, while retaining the safe `.env.example` template.

Hosted runs can incur API charges. Runbooks: `docs/griddle_llm.md` and `docs/pcg_maze_llm.md`.

Open challenge: agents that invent their own tools



How tools emerge

- LLaMEA and AlphaEvolve generate, mutate, evaluate and select code.
- LLM4AD modularizes tasks, search methods and evaluators.
- Claude Code and Codex can edit, run and debug whole repositories.

The harness must resist overfitting

- Visible development examples.
- Refreshed selection cases: seeds, maps, sizes and rules.
- Sealed final audit after the search is frozen.
- Quality, validity, runtime, robustness and diversity; sandbox all code.

Repeated feedback becomes an effective training signal. A credible result must transfer to untouched tasks.

LLaMEA · LLM4AD · AlphaEvolve · Codex · Claude Code

References

- Yao et al. (ICLR 2023), **ReAct: Synergizing Reasoning and Acting in Language Models**. Reasoning and action with observations.
- **MCP architecture**, specification 2025-06-18. Host, client and server responsibilities.
- Jiang, Earle, Khalifa and Togelius (2026), **Agentic PCG: Procedural Content Generation via Tool-using LLMs**. Project page; local PDF. Iterative level editing with tool and environment feedback.

Local reading copy and notes: `docs/papers/README.md`.

Code map

- `src/simple_examples/`: functions, server, MCP client, model loop.
- `src/game_agent/griddle/`: game, search server, LLM agent, benchmark.
- `docs/griddle_llm.md`: prompts, experiments and full report links.
- `src/pcg/maze/`: evolution, LLM and agentic generation; visualisation.
- `docs/pcg_maze*.md`: PCG commands, metrics and experiment reports.
- `docs/papers/`: local reading copies and connections to examples.

Build a useful tool. Connect a bounded loop. Measure the resulting behaviour.